

데이터 구조 설계

2차 과제 보고서

2022202109

이호정

Introduction

어떤 프로그램인가?

이 프로젝트는 항공편 관리 시스템이다.

B+ 트리, AVL 트리, STL 벡터와 같은 데이터 구조를 활용해서 항공편 데이터를 효율적으로 저장하고 관리할 수 있도록 설계되어있다. 주어진 명령어를 처리하면서 항공편 데이터를 삽입, 검색, 정렬, 출력하는 기능을 제공하며, 에러 처리와 로그 기록 또한 출력하는 프로그램이다. 해당 프로그램을 여러 조건에 맞게 구현하는 것이 이번 프로젝트의 목표이다.

main 기능에는 어떤 것들이 있는가?

파일 처리

input_data.txt: 항공편 정보를 담고 있는 파일로, 항공사 이름, 항공편 번호, 목적지, 좌석 수, 상태를 포함합니다.

command.txt: LOAD, ADD, SEARCH, PRINT 같은 명령어를 통해 항공편 데이터를 관리한다. 파일 형식을 검증하고 데이터를 읽어오는 과정을 처리합니다. 구현을 위해 여러 데이터 구조를 사용하였다.

B+ 트리는 항공편 데이터를 저장하고, 빠른 삽입과 검색, 범위 쿼리를 지원한다.

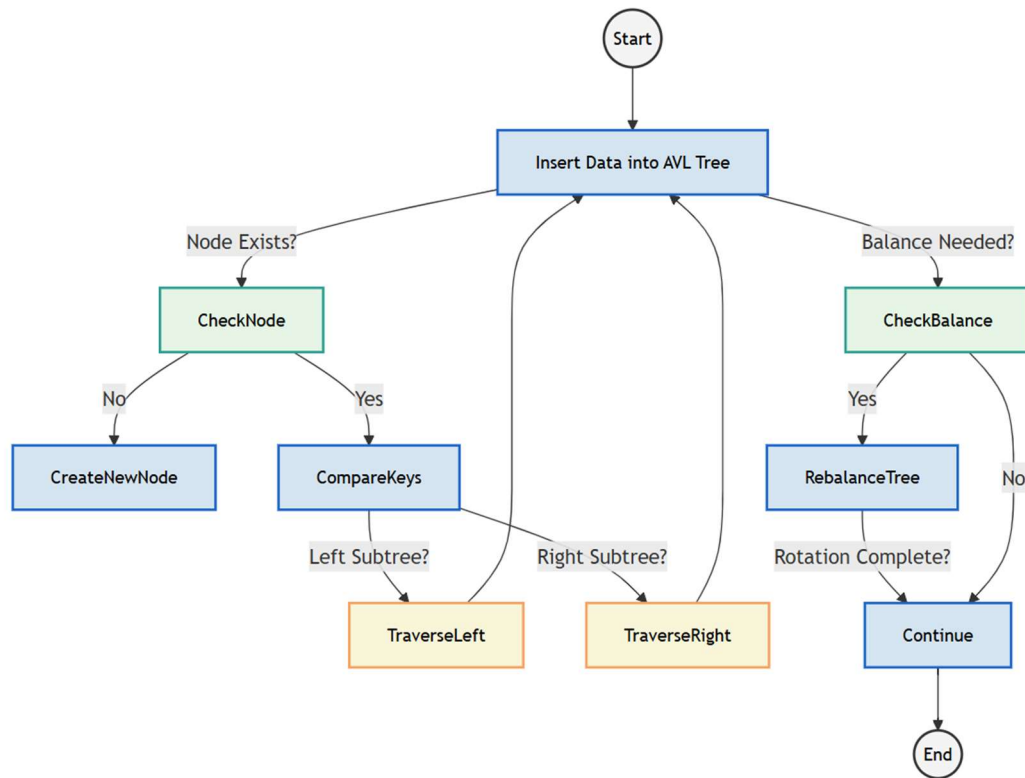
AVL 트리는 좌석 수가 0인 항공편 데이터를 관리하며, 트리 균형을 유지해 빠른 접근을 제공한다. STL 벡터는 데이터를 임시로 저장하고 정렬에 활용된다.

명령어

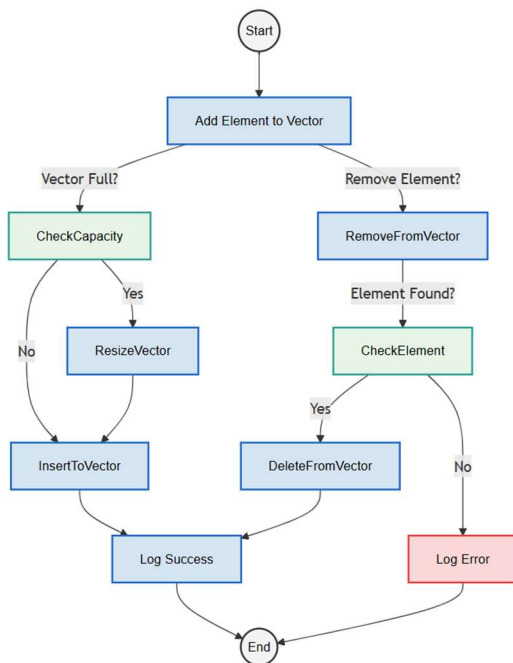
LOAD는 파일에서 데이터를 읽어 B+ 트리에 저장하는 기능이다. ADD는 새로운 항공편 데이터를 삽입하거나 기존 데이터를 수정하는 기능이다. SEARCH_BP는 특정 키 또는 범위를 기준으로 B+ 트리에서 데이터를 검색하는 기능이다. SEARCH_AVL은 AVL 트리에서 특정 항공편 데이터를 검색하는 기능이다. PRINT_BP는 B+ 트리에 저장된 모든 데이터를 출력하는 기능이다. VPRINT는 데이터를 정렬 기준에 따라 출력하는 기능이다.

Flowchart

● AVL tree 에 관한 전반적인 flowchart



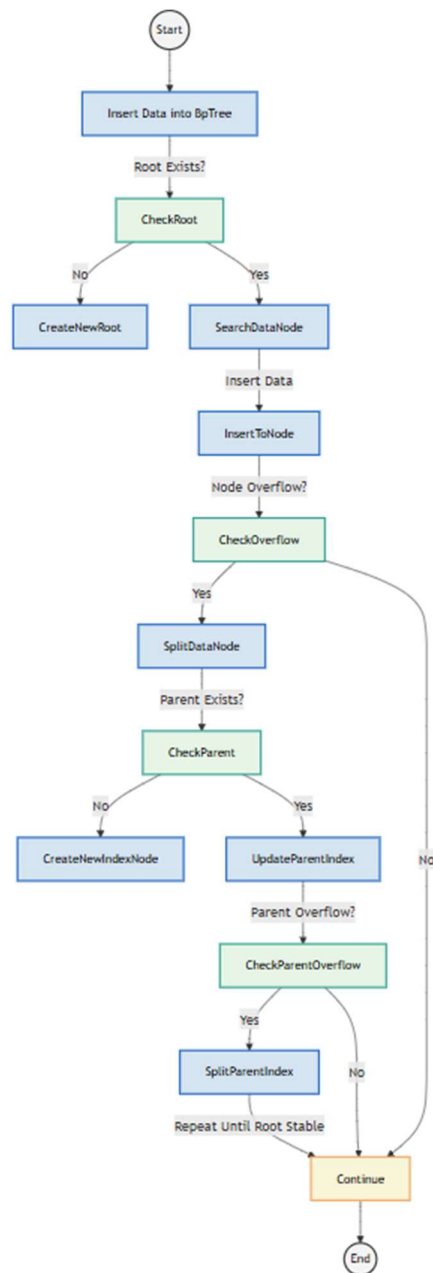
● STL Vector 에 관한 전반적인 flowchart



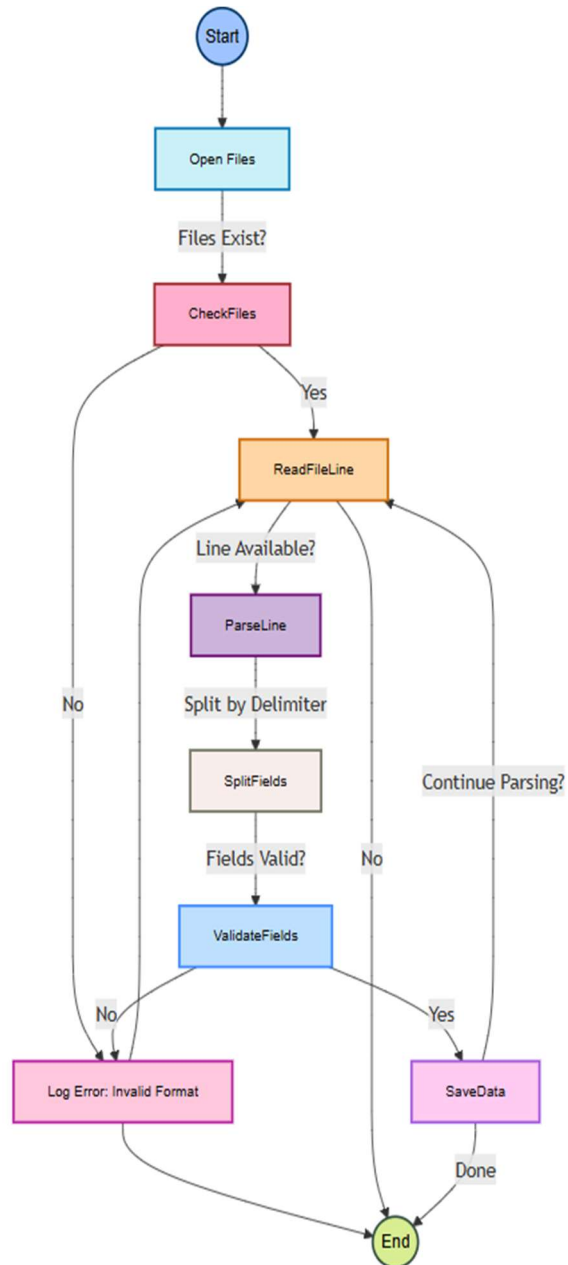
↓ AVL 트리에 데이터를 삽입하고 균형을 유지하는 과정을 설명합니다.

->Print 관련 명령어에서 데이터를 임시로 저장하거나 출력하는데 사용.

● B+ tree 의 전반적인 flowchart



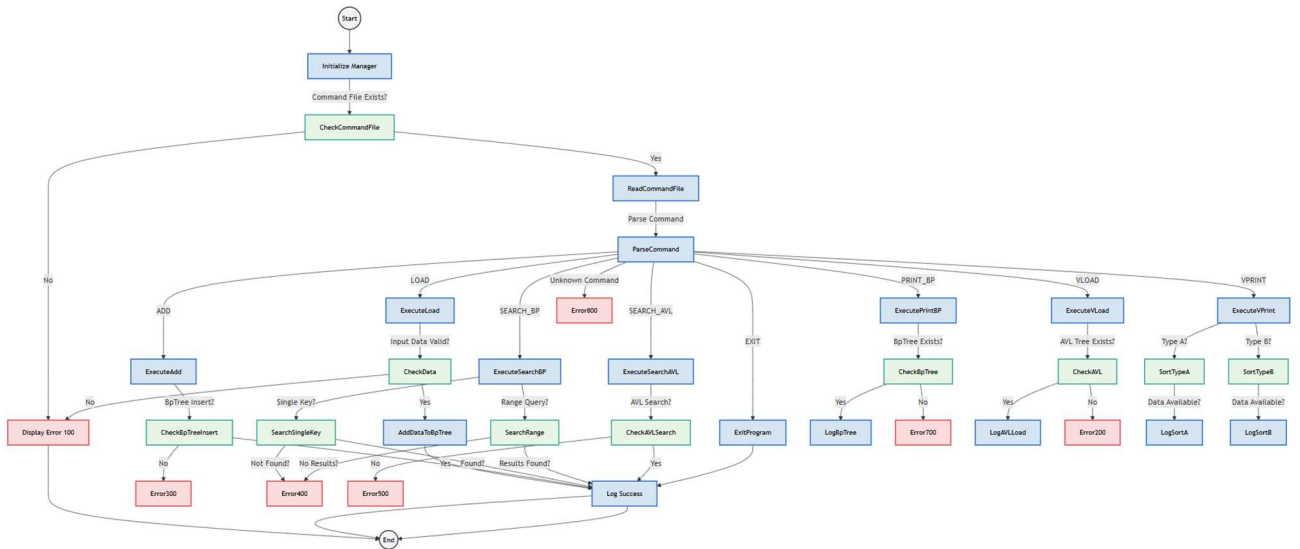
● Command Parsing의 flowchart



➔ 왼쪽 : 대규모 데이터를 정렬된 상태로 관리하며, Manager의 데이터 저장 명령 (LOAD, ADD)과 연동된다.

➔ 오른쪽 : 명령을 읽고 각각의 명령어로 스플릿 및 파싱하는 과정이다.

● 프로그램 전반적인 flowchart



➔ 전체 프로그램의 전반적인 컨트롤러로, 명령어에 따라 BpTree, AVL Tree, STL Vector 등을 호출하여 처리한다.

Algorithm

이 프로그램은 항공편 데이터를 효율적으로 관리하기 위해 B+ Tree와 AVL Tree 라는 두 가지 주요 데이터 구조를 활용한다.

1. File Parsing Algorithm

The program starts with parsing input data from input_data.txt and commands from command.txt. 데이터를 파싱하는 과정에서는 행을 기준으로 읽고, 공백(space) 또는 탭(tab)으로 구분된 데이터를 추출하여 객체로 변환한다. 파싱된 데이터는 유효성 검사를 거쳐 저장되며, 오류가 발견되면 특정 에러 코드를 출력한다.

해당 과정은 삽입 알고리즘 처럼 정형화 되어있는건 아니지만, 이번 과제를 하며 느낀게 많은 파트 중에 하나라 넣고 싶었다.

2. B+ Tree Insertion and Search Algorithm

B+ Tree는 프로그램의 핵심 데이터 구조로, 항공편 정보를 저장하고 검색한다.

Insertion: 새로운 항공편 데이터를 추가할 때 B+ Tree의 노드가 꽉 차면, split operation 이 수행된다. Split은 새로운 노드를 생성하고 기존 노드의 절반 데이터를 이동시킨다.

Search: 특정 항공편 번호(key) 또는 범위를 기반으로 데이터를 효율적으로 검색한다.

B+ tree 알고리즘

B+ 트리는 균형 잡힌 트리 구조로, 데이터베이스 인덱싱이나 파일 시스템 관리에 주로 사용된다. 데이터를 정렬된 상태로 유지하며 삽입, 삭제, 검색 작업을 효율적으로 처리할 수 있다. 데이터를 노드에 **삽입**하는 과정을 보이자면 다음과 같다.

새 데이터를 적절한 리프 노드에 삽입한다.

리프 노드가 초과되면 분할을 수행하고, 중간 키를 부모 노드로 올린다.

부모 노드가 초과되면 상위로 계속 분할을 진행하며, 필요 시 새로운 루트를 생성한다.

검색 : 루트에서 시작해 내부 노드를 따라가며 적절한 리프 노드에 도달한다. 데이터는 항상 리프 노드에 저장되므로 검색이 일관적이고 효율적이다.

삭제 : 데이터를 리프 노드에서 삭제하며, 노드가 부족할 경우 빌려오거나 *Merge을 수행한다. 이를 통해 트리의 균형을 유지한다.

B tree 와 b+ tree 의 차이점

B+ 트리는 데이터를 리프 노드에만 저장하고, 내부 노드에는 키 값만 저장한다. 반면, B 트리는 데이터가 리프 노드 뿐만 아니라 내부 노드에도 저장된다. 이로 인해 B+ 트리는 검색 시 모든 데이터를 리프 노드에서만 확인하기 때문에 일관성 있는 탐색 구조를 가지며, 범위 검색에서 리프 노드가 연결 리스트 형태로 연결되어 있어 더 효율적이다.

3. AVL Tree Algorithm

AVL Tree는 잔여 좌석이 없는 항공편 데이터를 저장하고 관리하기 위해 사용된다.

Insertion: 새로운 노드가 추가될 때 균형(balance)을 유지하기 위해 rotation이 수행된다.

Search: 특정 항공편을 검색할 때 AVL Tree는 균형 이진 트리의 구조를 활용하여 빠른 검색을 보장한다.

AVL 트리는 Binary Search Tree 의 업그레이드 버전으로, 삽입과 삭제 과정에서 항상 트리의 균형을 유지하는 게 특징인 트리이다.

노드 간의 Balance Factor를 계산해서, 높이의 차이를 나타내며, 이 차이가 -1, 0, 1 이내로 유지되도록 한다.

-삽입(Insertion) 알고리즘

먼저 트리에 새로운 데이터를 삽입한다.

그냥 일반적인 이진 탐색 트리처럼 새 데이터를 적절한 위치에 넣는다. 데이터가 크면 오른쪽, 작으면 왼쪽으로 내려가면서 빈 자리에 데이터를 꽂는다. 데이터를 삽입한 후에 각 노드의 균형을 확인한다. 균형 인수(Balance Factor)라는 걸 계산한다. 이건 그 노드의 왼쪽 서브트리 높이에서 오른쪽 서브트리 높이를 뺀 값이다. 이 값이 -1, 0, 1 사이면 트리는 여전히 균형 잡혀 있다. 딱히 더 할 일이 없다.

근데 만약 이 값이 -1, 0, 1의 범위를 벗어난다면?

이제 트리가 균형을 잃었다는 뜻이다. 그럼 네 가지 상황에 맞춰 회전을 해줘야 한다.

LL 상황

삽입된 데이터가 깨진 노드 기준 왼쪽-왼쪽으로 계속 내려간 경우다. 예를 들어, 30 밑에 20을 넣었고, 그 20 밑에 10을 넣었을 때가 이 경우다.

이럴 땐 깨진 노드를 중심으로 오른쪽으로 한 번 단순 회전(Rotate Right)을 하면 된다.

RR 상황

이건 LL의 반대다. 삽입된 데이터가 깨진 노드 기준 오른쪽-오른쪽으로 계속 내려간 경우다. 예를 들어, 10 밑에 20을 넣고, 20 밑에 30을 넣었을 때다.

이때 왼쪽으로 한 번 단순 회전(Rotate Left)을 한다

.

LR 상황

이건 조금 복잡하다. 삽입된 데이터가 깨진 노드 기준 왼쪽-오른쪽으로 내려간 경우다. 예를 들어, 30 밑에 10이 있고, 그 10의 오른쪽에 20을 넣었을 때다. 이때 두 번 회전이 필요하다. 먼저 깨진 노드의 왼쪽 자식을 중심으로 왼쪽으로 한 번 회전(Rotate Left)하고, 그다음 깨진 노드를 중심으로 오른쪽으로 한 번 더 회전(Rotate Right)하면 된다.

RL 상황

이건 LR의 반대다. 삽입된 데이터가 깨진 노드 기준 오른쪽-왼쪽으로 내려간 경우다. 예를 들어, 10 밑에 30이 있고, 그 30의 왼쪽에 20을 넣었을 때다.

이때 두 번 회전한다.

먼저 깨진 노드의 오른쪽 자식을 중심으로 오른쪽으로 한 번 회전(Rotate Right)하고,

그다음 깨진 노드를 중심으로 왼쪽으로 한 번 더 회전(Rotate Left)하면 된다.

이 과정을 거치면 트리는 다시 균형을 찾는다.

- 삭제(Deletion) 알고리즘

트리에서 데이터를 삭제한다.

이때도 일반적인 이진 탐색 트리 삭제 방법을 사용한다.

데이터를 삭제하면 빈 자리를 채우기 위해 왼쪽 서브트리에서 가장 큰 값을 가져오거나 오른쪽 서브트리에서 가장 작은 값을 가져오는 경우가 있다. 데이터 삭제 후 균형 인수를 확인한다. 삽입 때와 마찬가지로 각 노드의 균형 인수를 계산한다. -1, 0, 1 범위 안에 있다면 그냥 넘어간다.

균형이 깨진다면 회전을 수행한다.

삭제에서도 LL, RR, LR, RL 네 가지 상황이 있다.

삭제 이후 트리의 형태에 따라 삽입 때와 같은 방식으로 회전을 하면 된다.

-검색(Search) 알고리즘

검색은 그냥 평범하다. AVL 트리는 이진 탐색 트리와 동일하게 데이터를 검색한다. 루트에서 시작해서 찾고자 하는 값이 현재 노드보다 크면 오른쪽으로, 작으면 왼쪽으로 내려가면서 값을 찾는다. AVL 트리가 항상 균형을 유지하니까, 검색 속도는 $O(\log n)$ 으로 일정하다.

Result Screen

```
LOAD
ADDWtKoreanAirWtKE124WtSeoulWt7WtOn Time
SEARCH_BPWtKoreanAir
SEARCH_BPWtJEJUWtJeanAir
SEARCH_AVLWtASIANA
PRINT_BP
EXIT
```

```
LOAD
VLOAD
ADDWtKoreanAirWtKE123WtICNWt5WtBoarding
ADDWtASIANAWtOZ456WtLAXWt7WtDelayed
ADDWtJEJUWtJJ001WtCJUWt5WtBoarding
ADDWtJeanAirWtJA789WtBUSWt5WtDeparture
ADDWtAirKwangwoonWtAK101WtICNWt6WtDelayed
PRINT_BP
SEARCH_BPWtKE123
SEARCH_BPWtOZ456WtAK101
SEARCH_AVLWtKE123
VPRINTWtAVL
VPRINTWtFlightNumber
VPRINTWtDestination
```

->command.txt 예시

명령어 뒤에 인자가 오는 경우에는 wt 로 나누어준다.

LOAD 명령어

```
===== LOAD =====  
Success  
=====
```

```
===== ERROR =====  
100  
=====
```

→ 성공했을 때, 실패했을 때 이다.

동작 : LOAD 명령어 실행 후, 파일의 모든 항공권 정보가 B+ 트리와 AVL 트리에 저장

VLOAD -> AVL Tree에 저장된 모든 데이터 정보를 Print_vector에 불러오는 명령어

```
=====VLOAD=====  
Success  
=====
```

ADD

```
===== ADD =====  
OZ999 | ASIANA | MUC | 7 | Boarding  
=====
```

항공권 데이터를 추가하는 명령어 각 항공사의 조건에 따라 데이터를 B+ 트리에 삽입하거나 좌석 수를 업데이트한다. 좌석 수가 0인 항공권은 AVL 트리에 저장됨

PRINT_BP

```
=====PRINT_BP=====  
KoreanAir | KE123 | ICN | 5 | Boarding  
ASIANA | OZ456 | LAX | 7 | Delayed  
JEJU | JJ001 | CJU | 5 | Boarding  
JeanAir | JA789 | BUS | 5 | Departure  
AirKwangwoon | AK101 | ICN | 6 | Delayed  
=====
```

동작 : B+ 트리를 중위 순회하여 항공권 데이터를 출력한다.

SEARCH_BP

```
SEARCH_BPWtKE123
```

```
SEARCH_BPWtOZ456WtAK101 명령어가 이럴 경우
```

```
=====SEARCH_BP=====
KoreanAir | KE123 | ICN | 5 | Boarding
=====
```

-> SEARCH_BP 명령어는 B+ Tree에 저장된 데이터를 검색하는 명령어로 1개 또는 2개의 인자를 추가로 입력한다. EARCH_BP 명령어의 인자로 1개가 입력되는 경우, 해당 항공편의 검색 결과를 출력한다. SEARCH_BP 명령어의 인자로 2개(시작 단어, 끝 단어)가 입력되는 경우, 범위 검색의 결과를 출력한다

SEARCH_AVL

```
===== SEARCH_AVL =====
7C024 | JEJU | CJU | 0 | Delayed
=====
```

-> SEARCH_AVL 명령어에 인자로 항공편명을 입력해 AVL Tree에 저장되어 있는 이름에 대한 데이터만을 출력한다

VPRINT

```
===== VPRINT A =====
KoreanAir | KE123 | ICN | 5 | Boarding
ASIANA | OZ456 | LAX | 7 | Delayed
JEJU | JJ001 | CJU | 5 | Boarding
JeanAir | JA789 | BUS | 5 | Departure
AirKwangwoon | AK101 | ICN | 6 | Delayed
=====
```

-> Print_vector에 저장된 데이터를 조건 A, B에 따라 정렬하고

출력한다. 오름차순, 내림차순, 항공사명을 기준으로 정렬하고 동점일시 우선위가 더높은 것부터 출력한다.

PRINT_BP

```
===== PRINT_BP =====  
Success  
=====
```

-> B+ Tree에 저장된 데이터를 항공편명을 기준으로

오름차순으로 전부 출력

EXIT

```
===== EXIT =====  
Success  
=====
```

-> 프로그램 상의 메모리를 해제하며, 프로그램을 종료한다

Consideration

이번 프로젝트는 데이터 구조의 실제 활용을 체험할 수 있는 좋은 기회였다. 특히 AVL 트리와 B+ 트리를 직접 설계하고 구현하면서 각 자료구조의 장단점을 명확히 이해하게 되었다. 이를 기반으로 항공권 정보를 효과적으로 관리하고 검색하는 시스템을 설계하는 과정에서 다양한 도전을 경험했다.

프로젝트 초반, AVL 트리와 B+ 트리를 각각 다른 목적에 맞게 사용하는 설계를 요구받았다. AVL 트리는 좌석이 없는 항공편을 효율적으로 관리하기 위해 사용되었으며, B+ 트리는 많은 항공권 데이터를 정렬하고 검색하는 데 적합했다. 이 과정에서 자료구조 선택이 프로그램의 성능과 유지 보수성에 얼마나 중요한지 다시 한번 깨달았다. AVL 트리의 균형 유지 알고리즘과 B+ 트리의 노드 분할/합병 과정을 구현하며 기본 개념을 실질적으로 익힐 수 있었다.

명령어 기반 시스템을 설계하는 과정에서 LOAD, ADD, SEARCH 등의 명령어가 서로 복잡하게 연결되어 있었다. 특히, B+ 트리와 AVL 트리 간 데이터를 동기화하는 부분에서 에러 처리가 까다로웠다. 예를 들어, 좌석 수가 0이 되는 조건에서 B+ 트리에서 데이터를 삭제하고 AVL 트리에 추가하는 과정을 명확히 정의하지 않으면 데이터 일관성이 깨질 수 있었다. 이러한 문제를 해결하기

위해 각 명령어의 입력 조건과 실행 단계를 철저히 점검하고 명확히 설계했다.

프로젝트 중간쯤에 한계에 부딪혔을 때는 정말 포기하고 싶었다. 특히, 좌석 수와 상태 정보를 업데이트하는 과정에서 복잡한 조건을 처리하다 보니, 코드는 자꾸 꼬이고 원하는 대로 작동하지 않았다. 반복되는 디버깅과 예외 처리 작업에 지치고, 어디서부터 잘못된 것인지 감조차 오지 않아 스스로에 대한 의문이 생기기도 했다. 하지만 조금씩 문제를 나눠 해결하고, 한 가지씩 기능을 성공적으로 구현해 나가면서 결국 모든 요구사항을 충족시킬 수 있었다. 포기하지 않고 끝까지 해냈다는 점에서 큰 뿌듯함을 느꼈다.

항공권 데이터가 많아질수록 검색 및 삽입의 성능이 중요한 요소로 다가왔다. AVL 트리의 경우, 트리 균형을 유지하기 위한 회전 연산이 빈번하게 발생해 삽입 속도가 느려질 수 있었다. 반면, B+ 트리는 노드 크기와 차수를 조정하며 메모리 사용량과 성능 간 균형을 맞춰야 했다. 이러한 트레이드오프를 고려하며 코드 최적화에 신경을 썼다.

프로그램 동작 중 다양한 에러 시나리오를 처리해야 했다. 잘못된 명령어 입력, 데이터 파일 누락, 비어 있는 AVL 트리나 B+ 트리에서 데이터를 검색하려는 시도 등 모든 가능성을 고려해 에러 메시지를 명확히 출력하고 로그 파일을 기록하는 기능을 추가했다. 디버깅 과정에서는 오류의 원인을 빠르게 추적하기 위해 각 함수의 상태를 출력하거나 중간 결과를 점검했다.

개인적으로는 시간이 넉넉치 않아 아쉬움이 많이 남았다. 제안서에 나온 것들 이외에도 예외처리나 특이 케이스들을 수행하는 프로그램으로 만들 수 있었으면 좋았을 것 같다. 과제가 끝나고도 이부분에 대해서는 디벨롭 해보려한다.

이번 프로젝트는 단순히 프로그램을 완성하는 것을 넘어, 데이터 구조와 알고리즘의 실제적 중요성을 체감하고 깊이 이해할 수 있는 소중한 경험이었다. 특히, 어려운 문제를 극복하고 결과물을 완성했을 때 느낀 성취감은 앞으로의 개발 과정에서도 큰 동기부여가 될 것이다.